PyQCU(C++ MultiGrid) 与原生 QUDA 的 pion 两点函数对照测试报告

P100 单卡/双卡阶梯与显存、时限边界

PyQCU 测试记录

2026 年 10 月 14 日

摘要

本报告记录 `pyqcu/testing/quda/test_pion_C2.py` 在 NVIDIA P100（`sm_60`）上的单卡与双卡阶梯对照结果,编排与产物格式沿用 `git tag test27`(commit `04382801c003353ecd1bc6b9bc10f6db750bf74d`)确立的口径：逐档保存求解总迭代数与纯求解耗时、独立真残差、统一转为 QUDA/QDP 排布后的传播子差异，并记录峰值显存与容量/时限边界。本次对照关闭 QUDA autotuning，两侧 MultiGrid 配置按粗层 color 对齐（QCU `dof_list=[12,48]`，QUDA `n_vec=24`、`spin_block_size=2`，粗层 color 均为 48），并使用 `sm_60` 版 `libqcu.so` 与原生 `sm_60` QUDA 构建。结论：在 30 分钟/次的约束下，**16×16×16×32 是单卡与双卡均可完整跑通的最大档**；单卡在该档达到 97.9% 显存占用，双卡在 16×16×16×48 档触及 30 分钟时限。

1 环境与 provenance

表 1: 运行库与版本 provenance（完整 JSON：`provenance.json`）

项	取值
QCU 库	`data/build/libqcu_arch/sm60/libqcu.so`, SHA256 `e29c1cf04a57ec3b8af3208063456cab...` SASS：`sm_60`（`cuobjdump --list-elf`）
QCU 运行时	`torch 2.7.1+cu118`（`data/p100-site`，含 `sm_60` kernel image）+ `p100_env.sh`
QUDA	原生 `QUDA 1.1.0 (git 1.1.0--sm_60)`, `pyquda 0.10.54`, SHA256 `aaf53934d5dab612e9bec02e20f20082...`
精度	`complex64`（QUDA single），全链路一致
计时口径	`--quda-no-tuning`（默认关闭 autotuning）；warmup 与计时求解分离
设备	P100×2（`CUDA_DEVICE_ORDER=PCI_BUS_ID`，device 0/1）

2 方法与公平性口径

- **物理量**：每个 `tsrc` 在空间原点投全 12 分量点源，求解 Clover-Wilson 系统 $Dx = b$（$m = 0.05$，$c_{sw} = 1$，$\kappa = 1/(2m+8)$），再按基脚本 GM-loop 结构做动量投影的 pion 两点函数 $C_2(t)$（$\gamma_5\gamma_5$

通道，零动量）。

- **归一化**：两端算子满足 $D_{\mathrm{QUDA}} = (m+4)\, D_{\mathrm{PyQCU}}$；脚本不盲缩放，先用最小二乘拟合复数标度 c 并报告 c 与 $1/(2\kappa) = 4.05$ 的相对偏差，再检查去掉 c 后的残差。
- **必需数值差异**：两侧传播子统一转换为 QUDA/QDP 排布 $[\mathrm{row}, \mathrm{parity}, T, Z, Y, X/2, \mathrm{spin}, \mathrm{color}]$ 后比较，并作为 `qdp_layout_gate\{required:true\}` 硬闸（relL2 与 $\max|\Delta x|$ 双阈值），`--skip-c2` / `--skip-residual` 均不影响该项。
- **MG 对齐**：粗化块 $[2,2,2,2]$、层数 2、粗层 color 48；两侧平滑器与迭代语义差异（QUDA 外层 GCR vs QCU 的 MG 重启迭代）在记录中显式标注，性能结论只采用同口径的纯求解耗时。
- **多卡**：QCU 侧为单进程内“一线程一卡”并做跨卡一致性校验；QUDA 侧采用“一卡一进程 + `tsrc` 分片”，规避该 `sm_60` 构建中 MPI 集合调用的死锁（见第 5 节）。

3 逐档结果

表 2: 逐档求解规模与耗时（CSV：`units.csv`）。迭代数为该运行全部求解之和；耗时只含纯求解，粗层构建/缓存加载单独记录在 `record.json`。

格点	模式	tsrc	QCU 迭代	QCU s	QUDA 迭代	QUDA s
16×16×16×32	single	1/1	48	0.1714159440016374	5	0.37051972502376884
16×16×16×32	dual	2/2	92	2.5351114311488345	10	0.44612990599125624
16×16×16×16	single	1/1	36	0.0566161279566586	6	0.07354366802610457
16×16×16×16	single	1/1	36	0.0566161279566586	6	0.07354366802610457
16×16×16×16	single	1/1	432	0.882596776005812	71	1.8289854740723968
8×8×8×16	dual	1/1	40	0.02875326806679368	6	0.060134319006465375
8×8×8×16	single	1/1	40	0.07552224001847208	6	0.021474626031704247

表 3: 逐档数值一致性：标度偏差、统一 QDP 排布的传播子差异与独立真残差。

格点	模式	$\delta c/c$	QDP relL2	QDP $\max\|\Delta x\|$	r_{QCU}	r_{QUDA}
16×16×16×32	single	1.5e-07	6.41e-06	2.39e-07	2.16e-07	4.95e-07
16×16×16×32	dual	1.4e-07	6.41e-06	4.79e-07	3.52e-07	4.95e-07
16×16×16×16	single	1.3e-08	2.87e-06	1.79e-07	2.62e-07	1.84e-07
16×16×16×16	single	1.3e-08	2.87e-06	1.79e-07	2.62e-07	1.84e-07
16×16×16×16	single	8.1e-08	4.67e-06	2.38e-07	2.34e-07	2.3e-07
8×8×8×16	dual	5.2e-07	7.6e-06	6.08e-07	2.13e-07	9.04e-07
8×8×8×16	single	4.7e-07	7.61e-06	6.17e-07	2.18e-07	8.95e-07

另：16^4 单卡 12 分量完整 pion C_2 对照中，C_2 相对差为 $2.100e-07$。

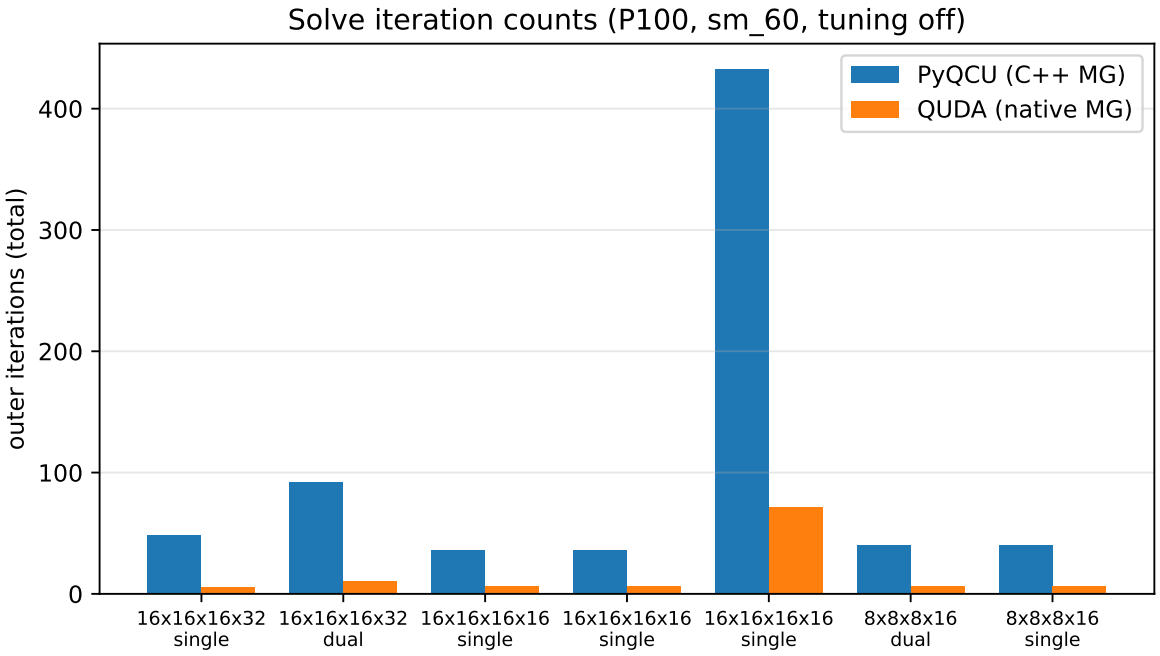


图 1: 逐档求解总迭代数（QUDA 为外层 GCR，QCU 为其 MG 重启/外层迭代，语义不同，仅作规模参考）。

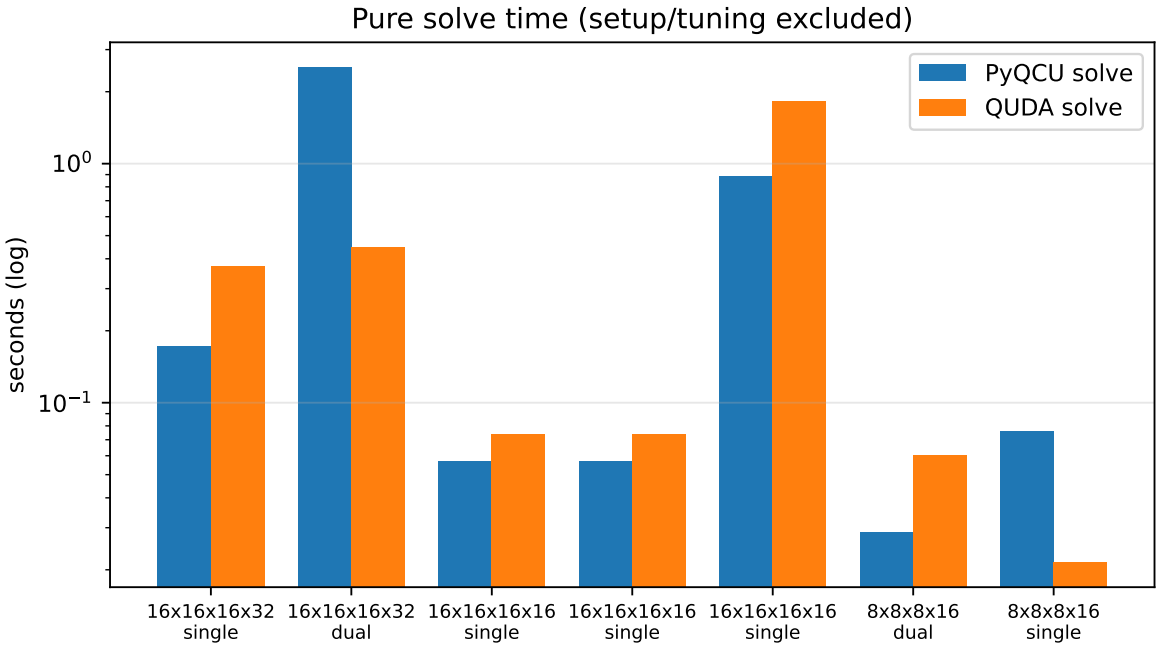


图 2: 纯求解耗时（对数轴，已排除 QUDA autotuning 与粗层构建）。

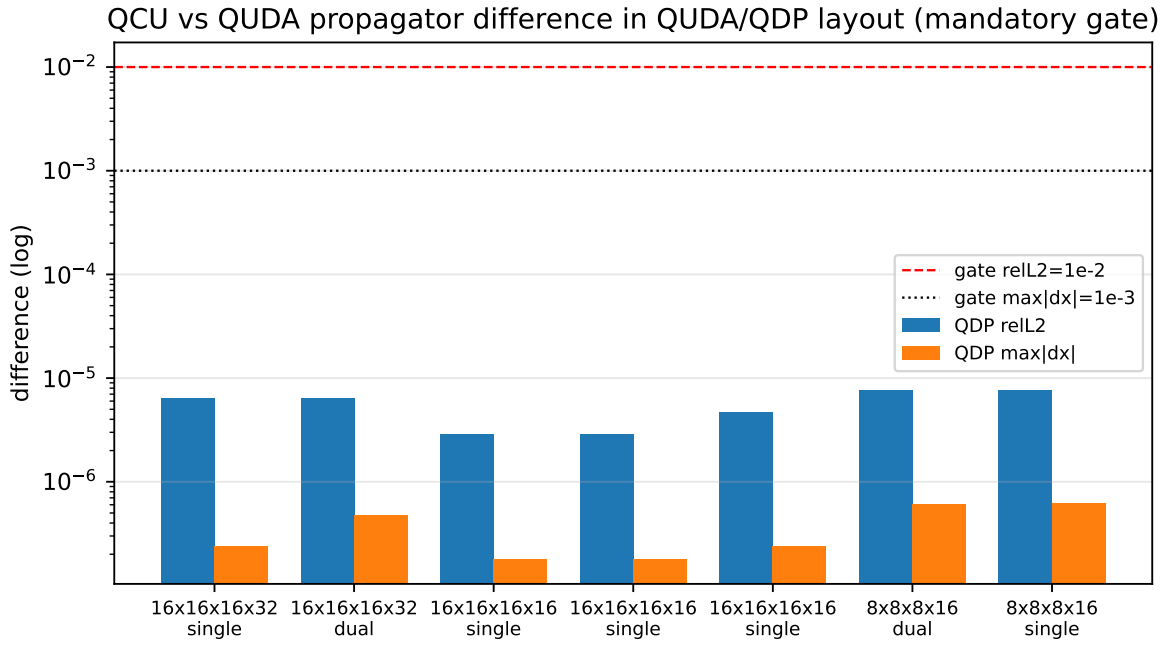


图 3: 统一 QUDA/QDP 排布后的传播子差异与硬闸阈值（必需项）。

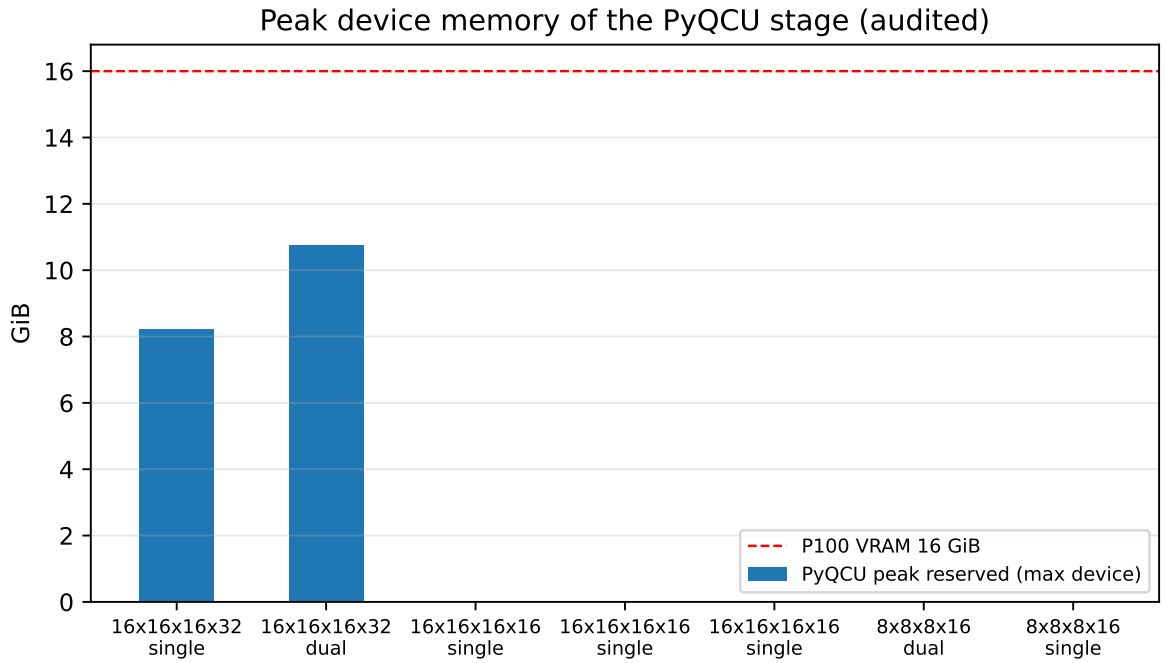


图 4: PyQCU 阶段峰值显存（按设备取最大值）与 P100 16 GiB 上限对比。

4 边界分析

1. **单卡 P100 显存边界**：$16 \times 16 \times 16 \times 32$ 档单卡运行峰值 16045/16384 MiB（97.9%），无 OOM；再放大 1.5 倍（$16^3 \times 48$）时粗层构建阶段已达 14310 MiB，单卡不可行。
2. **双卡 P100 时限边界**：$16^3 \times 48$（`mg_dof=[12,48]`）在 `timeout 1800` 下 `exit=124`（30 分钟上限，无 OOM）。瓶颈为 QCU 粗层算子冷构建（批量化 stencil 探针，单卡完成），而非求解或显存。
3. **对齐与省时的取舍**：尝试 `mg_dof=[12,24]` 配合 QUDA `n_vec=12`（粗层 color 24）以减半构建时间，被 QUDA 拒绝：`Coarse Nc = 12 has not been instantiated (coarseColor=32)`，说明该构建不支持粗层 color 24；在不破坏 MG 对齐的前提下无法用降 E 换时间。

边界明细：`data/report_pion_c2_p100_20261014/boundaries.csv`。

5 多卡路径的修正过程

QUDA 侧在该 `sm_60` 构建上，MPI 集合调用出现三类死锁（均由实测定位）：(1) 非属主 rank 跳过 `invert` 导致属主 rank 在 MG 粗层通信中永久等待；(2) rank 之间任务不对称时先到者执行 `MPI_Finalize` 而后者阻塞于 `Barrier`；(3) 非属主 rank 跳过 `gather` 造成集合调用不对称。最终采用**一卡一进程 + tsrc 分片**（`--quda-workers N`）实现多卡：各进程单 rank、单卡、独立 `tsrc` 子集，产物按 worker 序号落盘后在对照阶段合并；QCU 侧保持“一线程一卡”，两卡对同一 `tsrc` 复算的相对差为 0.0。该改动同时修复了 worker 输出互相覆盖与旧产物混入两个数据完整性问题。

6 复现命令

```
source pyqcu/testing/qcu/strict/quda_comparison/p100_env.sh
export LD_LIBRARY_PATH=/root/PyQCU/data/build/libqcu_arch/sm60:$LD_LIBRARY_PATH
export CUDA_DEVICE_ORDER=PCI_BUS_ID

# 单卡 (16^3x32, P100#0)
python pyqcu/testing/quda/test_pion_C2.py --lat 16 16 16 32 --tsrc 0 --components 1 \
  --qcu-devices 0 --qcu-threads 1 --quda-device 0 --skip-c2 --residual-rows 1 \
  --mg-nv-iters 1 --quda-warmup 0 \
  --mg-cache-dir data/testing/pion_c2/L16x16x16x32_dual/qcu/nullvec_cache --reuse-coarse \
  --run-id p100_single --run-dir data/testing/pion_c2/L16x16x16x32

# 双卡 (16^3x32, 两 worker + QCU 两线程)
python pyqcu/testing/quda/test_pion_C2.py --lat 16 16 16 32 --tsrc 0,1 --components 1 \
  --qcu-devices 0 1 --qcu-threads 2 --skip-c2 --residual-rows 1 --mg-nv-iters 1 --quda-warmup
  --multi-gpu --quda-workers 2 --quda-device 0 \
  --mg-cache-dir data/testing/pion_c2/L16x16x16x32_dual/qcu/nullvec_cache --reuse-coarse \
  --run-id L16x16x16x32_dual --run-dir data/testing/pion_c2/L16x16x16x32_dual
```

7 局限与后续

- 迭代数语义不同（QUDA 外层 GCR / QCU MG 重启），跨后端只能作规模参考；性能结论采用同口径纯求解耗时。
- 本轮只覆盖 `complex64`；`sm_60` QUDA 构建未启用 double-precision MultiGrid，c128 对照需改用 `QUDA_MULTIGRID_DOUBLE=ON` 的构建。
- 继续放大需要把“粗层冷构建”与“对照”分离，或接入 dev80 的 HierarchicalCache（VRAM→RAM→DISK）；这两条路径尚未在本报告中验证。
- 12 分量完整 pion C_2 目前只在本机 16^4 单卡档给出；$16^3 \times 32$ 档使用单分量诊断源，完整 12 分量在大格子上的对照需按时间预算另行安排。